

第6章 深度学习算子及模型实验

深度学习算子是构建神经网络模型的基本计算模块，涵盖卷积、归一化、激活、注意力等操作，直接决定了模型的表达能力与运行效率。高效的算子库不仅提升模型训练与推理性能，更是实现深度学习系统软硬件协同优化的关键。

第5章中我们基于高级语言 Triton 实践了算子开发和网络集成，本章将围绕 BCL (BANG C/C++ Language) 编程语言实现的高性能算子库 CNNL、CNNL-Extra、MLU-OPS、Torch-MLU-OPS 来介绍在不开发算子的情况下，如何利用各类高性能算子库在一款 DLP 硬件平台上快速搭建和部署深度学习神经网络。

首先，在实验开始前介绍 C++ 接口形式的 CNNL、CNNL-Extra、MLU-OPS 三个深度学习高性能算子库的基本功能与使用方式，然后介绍 Python 接口形式的 Torch-MLU-OPS 算子库。

随后，通过三层神经网络和 VGG19 图像分类网络两个实验，学习如何调用 C++ 接口的单算子库 CNNL 封装成的 Python 库实现 MLU 上的深度神经网络，通过 CPU 和 MLU 的网络对比实验，理解高性能算子库是如何配合异构 MLU 硬件实现深度神经网络加速的。理解了算子库的作用和构成后，第7章将深入算子库内部继续学习如何使用比 Triton 低一层级的 BCL 编程语言进行高性能算子开发。

最后，通过 Transformer 模型的推理实验，学习和理解大模型推理中的热点算子 Flash Attention 如何调用类似 Torch-MLU-OPS 这样的融合算子库快速部署和加速。

6.1 深度学习高性能算子库

在介绍深度学习高性能算子库前，我们先解答一个疑问：为什么 Triton 在开发效率和灵活性上表现出色，甚至能在特定操作上接近高性能算子库 cuDNN、cuBLAS 的性能，但 cuDNN、cuBLAS 在深度学习领域依然扮演着不可替代的角色？

首先，cuDNN、cuBLAS 作为 NVIDIA 官方推出的深度优化库，其稳定性和性能经过了无数工业场景的长期考验。对于追求极致性能和生产环境稳定性的企业来说，cuDNN、cuBLAS 提供的可靠保障是至关重要的。

其次，cuDNN、cuBLAS 支持的操作非常全面，涵盖了卷积、池化、激活函数、矩阵乘等各类深度学习核心操作。虽然 Triton 在矩阵乘法等操作上表现优异，但 cuDNN、cuBLAS 提供的这种“一站式”优化解决方案，对于构建复杂模型而言更为方便可靠。

最后，cuDNN、cuBLAS 与 GPU 硬件和软件生态（如 TensorRT）的深度集成，使其在利用最新硬件特性（如特定 Tensor Core 指令）方面可能仍有优势，这有时是通用编译器方案难以匹敌的。

简单来说，cuDNN、cuBLAS 这类深度学习高性能算子库像是性能稳定、Ahead-Of-Time (AOT) 编译和经过大量测试的“标准件”，而 Triton 则像是灵活多变、JIT 持续调优的“自定义工具”。两者并非简单的替代关系，而是满足不同需求的互补性工具。

接来了我们开始介绍 MLU 平台上和 cuDNN、cuBLAS 类似的几个深度学习高性能算子库：CNNL、CNNL-Extra、MLU-OPS、Torch-MLU-OPS。

6.1.1 CNNL 介绍

CNNL (Cambricon Neuware Neural Library) 是一个基于 MLU 硬件并针对深度神经网络的计算库, 针对人工智能领域的应用场景, 提供了高度优化的常用算子, 同时也为用户提供简洁、高效、通用、灵活并且可扩展的编程接口。MLU 硬件的高性能库 CNNL 用于在 MLU 上加速各种智能算法。用户可以直接调用 CNNL 中大量优化过的算子接口来实现其应用, 同时可以根据自己的需求扩展已有算子。下面从算法和数据结构两方面来了解 CNNL 算子库。

从算法视角看, CNNL 具有如下 4 方面功能:

1. 支持丰富的基本算子。
 - 常见的网络算子: 卷积、卷积反向求卷积输入与滤波的梯度; 池化、池化反向; 激活算子、激活算子反向, 如 ReLU、Sigmoid、Tanh 等; Softmax 前向与反向; Batchnorm 前向与反向、LayerNorm 前向与反向; Reduce 类算子。
 - 矩阵、计算类算子: 矩阵乘; 张量加、减、乘等基本运算; 张量变换, 如 Transpose、Split、Slice、Concat 等; 三角类变换, 如 Sin、Cos 等。
 - 循环网络算子: LSTM (Long Short-Term Memory); GRU (Gate Recurrent Unit)。
 - TensorFlow 和 PyTorch 常用算子: Embedding 前向和反向计算; NLLLoss 前向和反向计算。
2. 以通用为基本设计原则, 算子支持不同的数据布局、灵活的维度限制以及多样的数据类型。
3. 结合 MLU 的硬件架构特点, 极致优化算子性能, 并且尽最大可能地减少内存占用。
4. 提供包含资源管理的接口, 满足用户更多线程、多板卡的应用场景。

从数据结构视角看, CNNL 的编程模型里有如下 5 个核心概念:

1. 张量, 即具有统一数据类型的高维数组。张量描述符, 描述所指向的张量具体信息的载体, 包含了张量数据的数据布局、数据类型、维度个数、维度大小、步长大小等信息。通过张量描述符的表征, 每一块张量数据才得以具有实际的语义信息, 进而可以用来表示图像、语音或视频等各领域的数据。

- 数据布局: 表示没有维度语义的 array, 或者具有维度语义的布局, 如 NCHW, NHWC, HWCN, NDHWC, NCDHW。CNNL 使用 `cnnlTensorLayout_t` 来描述数据布局。

- 数据类型: 表示张量中数据的类型, 如 half, float, int8, int16 等。CNNL 使用 `cnnlDataType_t` 来描述数据类型。

- 维度个数: 针对数据布局为 array 的张量, 表示张量的维度, CNNL 支持 1-8 维。

- 维度大小: 表示每个维度的大小。

- 步长大小: 表示在同一个维度中访问两个相邻元素时需要走过的步数。举例: 我们定义一个 [3,4] 的二维矩阵 matrix, 默认情况下其具有隐含的步长信息数组 [4,1], 隐含着用户访问矩阵同维度内相邻元素需要间隔的元素个数信息, 步长数组 [4,1] 的 1 表示访问第二个维度相邻元素时 (例如 matrix[2][1] 和 matrix[2][2] 之间), 内存上的间隔是 0, 也就是步

长为 1；访问第一个维度相邻元素时（例如 `matrix[1][1]` 和 `matrix[2][1]` 之间），内存上的间隔是 3，也就是步长为 4。用户可以在 CNNL 提供的张量描述符的相关接口中显示地、灵活地配置步长数组，来对真实张量数据进行灵活地操作。

2. 句柄, 是指 CNNL 算子计算时通过句柄 (handle) 来维护 MLU 设备信息和队列信息等 CNNL 库运行环境的上下文。因此, 需要在使用 CNNL 库时, 创建句柄并将句柄绑定到使用的 MLU 设备和队列上。

- 用法: 用户需要先调用 `cnnlCreate()` 接口在初始化 CNNL 时创建一个句柄。创建的句柄随后会作为参数传递给算子相关接口完成数据计算。当 CNNL 库使用结束时, 用户需要调用 `cnnlDestroy()` 接口释放与 CNNL 库有关的资源。

- 绑定: 在同一线程中, `cnnlCreate()` 和 `cnnlDestroy()` 接口之间的调用只会使用与句柄绑定的 MLU 设备。如果想要使用多个 MLU 设备, 用户可以创建多个句柄来完成。通过使用句柄用户可以灵活地在多线程、多设备、多队列等多种场景下, 完成相应的计算任务。例如, 用户可以在多个线程中调用 `cnrtSetDevice()` 接口来设置使用不同的 MLU 设备, 同时调用 `cnnlCreate()` 接口创建多个句柄绑定到不同的 MLU 设备上, 再调用 CNNL 算子计算接口传入句柄, 从而实现在不同线程中, 算子的计算任务运行在不同的设备上。

3. 工作空间: CNNL 库中部分算子除了输入和输出数据的存放需要内存空间外, 还需要额外的内存空间用于算子计算的优化。这部分额外的内存空间被称为工作空间 (Workspace)。

4. 额外输入: CNNL 库中的部分算子除了算子语义上的输入和输出外, 还需要一个额外的输入内存空间 (ExtraInput) 用于算子的性能优化。ExtraInput 和正常的输入具有相同的特点。用户在使用带有 ExtraInput 参数的算子接口时, 需要将 MLU 设备端的 ExtraInput 指针传入算子接口中。CNNL 会提供相关的接口和操作机制来对 ExtraInput 指向的内存进行赋值, 用户只需要调用相关接口完成 ExtraInput 内存的数据赋值即可。

5. 算子描述符: 某些算子具备一些固有属性, 如 Convolution 算子在计算时需要的 Pad、Stride 等信息。CNNL 使用算子描述符 (Descriptor) 来描述算子的固有属性, 并提供相关接口创建、设置和销毁算子描述符, 从而简化算子计算接口中参数的数量。对于使用了算子描述符的算子, 描述符中包含的参数需要通过设置描述符传入算子计算接口, 而不是通过张量来设置。用户需要在调用算子计算接口前完成算子描述符的设置。

6.1.2 CNNL-Extra 介绍

CNNL-Extra 是一个基于 MLU 以及 CNNL 对 DNN 各类应用场景进行算子融合优化的计算库。在 CNNL 基础上, CNNL-Extra 针对 DNN 应用场景里的部分网络片段, 尤其是大语言模型网络, 提供了高度优化的融合算子。CNNL 和 CNNL-Extra 的接口形式和编译模型高度一致。

为什么 CNNL-Extra 和 CNNL 两个算子库不能合并成一个呢? 两个原因:

首先是实际应用中单算子的集合是相对固定的, PyTorch 等框架的性能基线一般是基于调用单算子的加速库得来, CNNL 提供相对稳定的单算子集合可以减少 API 的变动, 随着深度学习神经网络的发展, 尤其是大模型的快速演进, 更多的网络片段被整体优化成一个算子来提升性能, CNNL-Extra 可以快速扩展新增更多融合算子;

其次是考虑算子库的体积问题, 算子库一般是 AOT 编译出来的动态链接库, 随着算子

数量增长, 越来越大的动态库加载后一方面会占用更多的 CPU 内存, 另一方面算子的指令等数据也会占用更多的 DLP 设备端内存, 从而导致推理或训练可用的数据内存减少无法负载更大的模型。

产业界面对算子库体积侵蚀设备端内存的问题, 随即推出了延迟加载技术来优化算子占用的设备端内存, 这里不做深入讲解, 可以简单概括为只有当某个算子内的核函数真实被调用过, 才会触发 DLP 设备端真实分配内存来加载指令和数据, 这样算子库中没有被用的算子在实际应用中不会占用设备端内存。

CNNL-Extra 编程中所涉及的基本概念, 包括张量 (Tensor)、句柄 (Handle)、工作空间 (Workspace) 以及描述符 (Descriptor), 和 CNNL 的基本概念完全一致。CNNL-Extra 在编译时和运行时都直接依赖 CNNL 库提供的各类基础数据接口。用户为 CNNL 算子创建的输出 Tensor 描述符可以直接作为 CNNL-Extra 算子的输入。CNNL-Extra 算子使用的句柄 handle 所包含的对硬件资源的抽象也同 CNNL 保持一致。因此用户可以在构建网络毫无顾忌的按照实际需要混合使用 CNNL 和 CNNL-Extra 提供的算子接口。

CNNL-Extra 主要支持推理和训练领域的如下特性:

1. 支持常见网络片段的融合算子:

- 大语言模型网络推理场景: ScaledDotProductAttn 融合算子;

SingleQueryCachedKVAttn 融合算子; LLMQuantMatmul 融合算子; FuseNorm 融合算子;

- 大语言模型网络训练场景: FFlashAttnForward 融合算子; FlashAttnBackward 融合算子;

MaskedSoftmax 融合算子; RotaryEmbedding 融合算子;

- 检测类网络: YOLOv8 检测网络的后处理算子; YOLOX 检测网络的后处理算子;

• Transformer、Bert 网络: TransformerAttention 融合算子; TransformerAttnProj 融合算子;

TransformerFeedForward 融合算子;

- 搜索类网络: FlatSearch 融合算子。

2. 支持的大模型至少包括:

- LLaMA、Llama 2、Llama 3;
- ChatGLM、ChatGLM 2;
- BLOOM;
- Mixtral-8x7B;
- Stable Diffusion。

3. 算子支持不同的数据布局、灵活的维度限制以及多样的数据类型。

4. 结合 MLU 硬件架构特点, 优化融合算子, 使算子具有最佳性能, 并且尽最大可能减少内存占用。

5. 提供包含资源管理的接口, 满足用户多线程、多板卡的应用场景。

6.1.3 MLU-OPS 介绍

MLU-OPS 是 CNL 算子库的开源版本^①，MLU-OPS 旨在通过提供示例代码，供开发者参考使用，可用于开发自定义算子，实现对应模型的计算。

MLU-OPS 不仅仅提供了一些算子的 BCL 开源实现，还提供了一些算子开发和调试时常用的功能如下：

1. 算子精度标准；
2. 算子性能标准；
3. MLU-OPS 调用 CNL 算子方法；
4. 测试模块 GTest：支持内存泄露测试、代码覆盖率测试；
5. Gen-case：运行时测例生成工具；
6. Perf-Analyse：算子性能分析工具。

6.1.4 Torch-MLU-OPS 介绍

Torch-MLU-Ops 是针对 MLU 设计和开发的 PyTorch 第三方算子库。对于使用 PyTorch 框架的开发者，通过 Torch-MLU-Ops，能够便捷地使用这些自定义算子，进行算子的集成、评测和业务部署。

Torch-MLU-Ops 已全量覆盖 LLM（Large Language Model）推理场景下的常见算子。作为后端，已支持 vLLM、TGI、Stable Diffusion Web UI、ComfyUI 以及 Diffusers。

Torch-MLU-Ops 在支持 LLM 模型部署时提供的关键特性如下所示：

1. PyTorch 风格的算子 API；
2. 支持 LLM 推理场景下的常见算子；
3. Flash Attention 支持 Multi-head Attention、Multi-query Attention、Group-query Attention 特性；
4. 支持 KV Cache INT8 特性；
5. Single Query Cached Attention 算子支持 Paged 特性；
6. 矩阵乘类算子支持 Weight-only、AWQ、GPTQ、Smoothquant 量化特性；
7. 支持 MoE 场景下常见算子；
8. 提供不同融合力度的融合算子；
9. 提供通信与计算融合算子 MC2；
10. 支持 TGI 调用；
11. 支持 vLLM 调用；
12. 支持 Stable Diffusion Web UI 调用；
13. 支持 ComfyUI 调用；
14. 支持 Diffusers 调用。

^①MLU-OPS 开源地址见：<https://github.com/Cambricon/mlu-ops>

6.2 基于三层神经网络实现手写数字分类

6.2.1 实验目的

本实验旨在帮助学生掌握神经网络的基本设计原理与实现方法，理解深度学习算子的作用及其在异构平台上的应用。通过构建一个三层全连接神经网络模型完成手写数字分类任务，学生将系统掌握神经网络从建模、训练到推理的全过程，能够在 CPU 平台上手动实现训练流程，并在 MLU 平台上完成推理迁移，从而初步具备在异构平台上部署神经网络的能力。具体目标包括：

1) 掌握三层神经网络的组成及其工作原理，理解全连接层、激活函数、损失函数等模块在前向传播与反向传播中的作用，理清各模块之间的依赖关系，为构建更复杂的神经网络模型奠定理论基础。

2) 能够使用 Python 语言实现三层全连接神经网络在 CPU 平台上的训练与推理，理解梯度下降算法的基本机制，掌握模型参数的优化过程，加深对训练数据、损失函数与网络输出之间关系的理解。

3) 学会调用封装好的 python 算子库接口，在 MLU 平台上实现训练好的神经网络模型的推理功能，了解异构平台下模型移植的基本流程及注意事项，初步掌握算子级别的模型部署方式。

4) 通过实验结果对比，理解深度学习处理器在推理性能方面的优势与局限，提升对异构计算架构的感性认识，为后续高性能深度学习应用的优化与平台选择提供实践经验。

实验工作量：约 40 行代码，约需 4 个小时。

6.2.2 背景知识

6.2.2.1 神经网络的组成

一个完整的神经网络通常由多个基本的网络层堆叠而成。本实验中的三层神经网络由三个全连接层构成，在每两个全连接层之间插入 ReLU 激活函数引入非线性变换，最后使用 Softmax 层计算交叉熵损失函数，如图6.1所示。本实验中使用的的基本单元包括全连接层、ReLU 激活函数、Softmax 损失函数，将在本节分别介绍。更多关于神经网络中基本单元的介绍详见《智能计算系统》教材^[77]第 2.3 节。

1) 全连接层

全连接层以一维向量作为输入，输入与权重相乘后再与偏置相加得到输出向量。假设全连接层的输入为一维向量 $\mathbf{x}$ ，维度为 m ；输出为一维向量 $\mathbf{y}$ ，维度为 n ；权重是二维矩阵 $\mathbf{W}$ ，维度为 $m \times n$ ，偏置是一维向量 $\mathbf{b}$ ^①，维度为 n 。前向传播时，全连接层的输出的计算公式为

$$\mathbf{y} = \mathbf{W}^T \mathbf{x} + \mathbf{b} \quad (6.1)$$

在计算全连接层的反向传播时，给定神经网络损失函数 L 对当前全连接层的输出 $\mathbf{y}$ 的偏导 $\nabla_{\mathbf{y}} L = \frac{\partial L}{\partial \mathbf{y}}$ ，其维度与全连接层的输出 $\mathbf{y}$ 相同，均为 n 。根据链式法则，全连接层的权

^①偏置可以是一维向量，计算每个输出使用不同的偏置值；偏置也可以是一个标量，计算同一层的输出使用同一个偏置值。

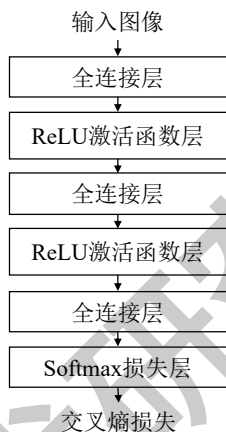


图 6.1 用于手写数字分类的三层全连接神经网络

重和偏置的梯度 $\nabla_{\mathbf{W}} L = \frac{\partial L}{\partial \mathbf{W}}$ 和 $\nabla_{\mathbf{b}} L = \frac{\partial L}{\partial \mathbf{b}}$ ，以及损失函数对输入的偏导 $\nabla_{\mathbf{x}} L = \frac{\partial L}{\partial \mathbf{x}}$ 的计算公式分别为：

$$\begin{aligned}\nabla_{\mathbf{W}} L &= \mathbf{x} \nabla_{\mathbf{y}} L^T \\ \nabla_{\mathbf{b}} L &= \nabla_{\mathbf{y}} L \\ \nabla_{\mathbf{x}} L &= \mathbf{W}^T \nabla_{\mathbf{y}} L\end{aligned}\quad (6.2)$$

实际应用中通常使用批量 (batch) 随机梯度下降算法进行反向传播计算，即选择若干个样本同时计算。假设选择的样本量为 p ，此时输入变为二维矩阵 $\mathbf{X}$ ，维度为 $p \times m$ ，每行代表一个样本。输出也变为二维矩阵 $\mathbf{Y}$ ，维度为 $p \times n$ 。此时全连接层的前向传播计算公式由公式(6.1)变为

$$\mathbf{Y} = \mathbf{XW} + \mathbf{b} \quad (6.3)$$

其中的 $+$ 代表广播运算，表示偏置 $\mathbf{b}$ 中的元素会被加到 $\mathbf{XW}$ 的乘积矩阵对应的一行元素中。权重和偏置的梯度 $\nabla_{\mathbf{W}} L$ 、 $\nabla_{\mathbf{b}} L$ 以及损失函数对输入的偏导 $\nabla_{\mathbf{x}} L$ 的计算公式由公式(6.2)变为

$$\begin{aligned}\nabla_{\mathbf{W}} L &= \mathbf{X}^T \nabla_{\mathbf{Y}} L \\ \nabla_{\mathbf{b}} L &= \mathbf{1} \nabla_{\mathbf{Y}} L \\ \nabla_{\mathbf{x}} L &= \nabla_{\mathbf{Y}} L \mathbf{W}^T\end{aligned}\quad (6.4)$$

其中计算偏置的梯度 $\nabla_{\mathbf{b}} L$ 时，为确保维度正确，用 $\nabla_{\mathbf{Y}} L$ 与维度为 $1 \times p$ 的全 1 向量 $\mathbf{1}$ 相乘。

2) ReLU 激活函数层

ReLU 激活函数是按元素运算操作，其输出向量 $\mathbf{y}$ 的维度与输入向量 $\mathbf{x}$ 的维度相同^①。在前向传播中，如果输入 $\mathbf{x}$ 中的元素值小于 0，则输出为 0，否则输出等于输入。因此 ReLU 的计算公式为

$$\mathbf{y}(i) = \max(0, \mathbf{x}(i)) \quad (6.5)$$

其中 $\mathbf{x}(i)$ 和 $\mathbf{y}(i)$ 分别代表 $\mathbf{x}$ 和 $\mathbf{y}$ 在位置 i 的值。

^①输入和输出也可以是矩阵，处理方式类似。

由于 ReLU 激活函数不包含参数，在反向传播计算过程中仅需根据损失函数对输出的偏导 $\nabla_{\mathbf{y}} L$ 计算损失函数对输入的偏导 $\nabla_{\mathbf{x}} L$ 。损失函数对本层的第 i 个输入的偏导 $\nabla_{\mathbf{x}(i)} L$ 的计算公式为

$$\nabla_{\mathbf{x}(i)} L = \begin{cases} \nabla_{\mathbf{y}(i)} L, & \mathbf{x}(i) \geq 0 \\ 0, & \mathbf{x}(i) < 0 \end{cases} \quad (6.6)$$

3) Softmax 损失层

Softmax 损失层是目前多分类问题中最常用的损失函数层。假设 Softmax 损失层的输入为向量 $\mathbf{x}$ ，维度为 k 。其中 k 对应分类的类别数，如对手写数字 0 至 9 进行分类时，类别数 $k = 10$ 。

在前向传播的计算过程中，对 $\mathbf{x}$ 计算 e 指数并进行归一化，即得到 Softmax 分类概率。假设 $\mathbf{x}$ 在位置 i 的值为 $\mathbf{x}(i)$ ， $\hat{\mathbf{y}}(i)$ 为 Softmax 分类概率 $\hat{\mathbf{y}}$ 在位置 i 的值， $i \in [1, k]$ 且为整数，则 $\hat{\mathbf{y}}(i)$ 的计算公式为

$$\hat{\mathbf{y}}(i) = \frac{e^{\mathbf{x}(i)}}{\sum_j e^{\mathbf{x}(j)}} \quad (6.7)$$

在神经网络推理时，完成 Softmax 损失层的前向传播之后，取 Softmax 分类概率 $\hat{\mathbf{y}}$ 中最大概率对应的类别作为预测的分类类别。

在神经网络训练时，在上述前向传播基础上，Softmax 损失层还需要根据给定的标记 (label，也称为真实值或实际值) 计算总的损失函数值。在分类任务中，标记通常表示为一个维度为 k 的 one-hot 向量 $\mathbf{y}$ ，该向量中对应真实类别的分量值为 1，其他值为 0。Softmax 损失层使用交叉熵损失函数 L ，其计算公式为

$$L = - \sum_i \mathbf{y}(i) \ln \hat{\mathbf{y}}(i) \quad (6.8)$$

在反向传播的计算过程中，对于 Softmax 损失层，损失函数对输入的偏导 $\nabla_{\mathbf{x}} L$ 的计算公式为

$$\nabla_{\mathbf{x}} L = \frac{\partial L}{\partial \mathbf{x}} = \hat{\mathbf{y}} - \mathbf{y} \quad (6.9)$$

由于工程实现中使用批量随机梯度下降算法，假设选择的样本量为 p ，Softmax 损失层的输入变为二维矩阵 $\mathbf{X}$ ，维度为 $p \times k$ ， $\mathbf{X}$ 的每个行向量代表一个样本，则对每个输入计算 e 指数并进行行归一化得到

$$\hat{\mathbf{Y}}(i, j) = \frac{e^{\mathbf{X}(i, j)}}{\sum_k e^{\mathbf{X}(i, k)}} \quad (6.10)$$

其中 $\mathbf{X}(i, j)$ 代表 $\mathbf{X}$ 中对应第 i 个样本 j 位置的值。当 $\mathbf{X}(i, j)$ 数值较大时，求 e 指数可能会出现数值上溢的问题。因此在实际工程实现时，为确保数值稳定性，会在求 e 指数前先行减最大值处理，此时 $\hat{\mathbf{Y}}(i, j)$ 的计算公式变为

$$\hat{\mathbf{Y}}(i, j) = \frac{e^{\mathbf{X}(i, j) - \max_n \mathbf{X}(i, n)}}{\sum_k e^{\mathbf{X}(i, k) - \max_n \mathbf{X}(i, n)}} \quad (6.11)$$

在神经网络推理时，取 Softmax 分类概率 $\hat{\mathbf{Y}}(i, j)$ 的每个样本，即每个行向量中最大概率对应的类别作为预测的分类类别。

做神经网络训练时, 标记通常表示为一组 one-hot 向量 $\mathbf{Y}$, 维度为 $p \times k$, 其中每行是一个 one-hot 向量, 对应一个样本的标记。则计算损失值的公式(6.8)变为

$$L = -\frac{1}{p} \sum_{i,j} Y(i, j) \ln \hat{Y}(i, j) \quad (6.12)$$

其中损失值是所有样本的平均损失, 因此对样本数量 p 取平均。

在反向传播时, 当选择的样本量为 p 时, 损失函数对输入的偏导 $\nabla_{\mathbf{x}} L$ 的计算公式(6.9)变为

$$\nabla_{\mathbf{x}} L = \frac{1}{p} (\hat{\mathbf{Y}} - \mathbf{Y}) \quad (6.13)$$

6.2.2.2 神经网络训练

神经网络训练通过调整网络层的参数来使神经网络计算出来的结果与真实结果 (即标记) 尽量接近。神经网络训练通常使用随机梯度下降算法, 通过不断地迭代计算每层参数的梯度, 利用梯度对每层参数进行更新。具体而言, 给定当前迭代的训练样本, 其中包含输入数据及标记信息, 首先进行神经网络的前向传播处理, 即输入数据和权重相乘再经过激活函数计算出隐层, 隐层神经元与下一层的权重相乘再经过激活函数得到下一个隐层, 通过逐层迭代计算出神经网络的输出结果。随后利用输出结果和标记信息计算出损失函数值。然后进行神经网络的反向传播处理, 从损失函数开始逆序逐层计算损失函数对权重和偏置的偏导 (即梯度), 最后利用梯度对相应的参数进行更新。更新参数 $\mathbf{W}$ 的计算公式为

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} L \quad (6.14)$$

其中, $\nabla_{\mathbf{W}} L$ 为参数的梯度, η 是学习率。

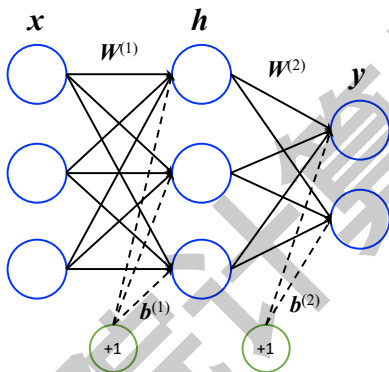


图 6.2 两层神经网络示例

下面以图6.2中的两层神经网络为例, 介绍神经网络训练的具体过程。图6.2中的网络由两个全连接层及 Softmax 损失层组成^①, 其中第一个全连接层的权重为 $\mathbf{W}^{(1)}$, 偏置为 $\mathbf{b}^{(1)}$, 第二个全连接层的权重为 $\mathbf{W}^{(2)}$, 偏置为 $\mathbf{b}^{(2)}$ 。假设某次迭代的神经网络输入为 $\mathbf{x}$, 对应的

^①本实验中实现的三层神经网络与这个例子非常类似, 即在这个例子基础上再添加一层全连接层和一层 ReLU 层即可, 网络训练的过程也与这个例子完全一致

标记为 $\mathbf{y}$ 。该神经网络前向传播的逐层计算公式依次是

$$\begin{aligned}\mathbf{h} &= \mathbf{W}^{(1)T} \mathbf{x} + \mathbf{b}^{(1)} \\ \mathbf{z} &= \mathbf{W}^{(2)T} \mathbf{h} + \mathbf{b}^{(2)}\end{aligned}\quad (6.15)$$

其中 $\mathbf{h}, \mathbf{z}$ 分别是第一、第二层全连接层的输出。Softmax 损失层的损失值 L 为：

$$\begin{aligned}\hat{\mathbf{y}}(i) &= \frac{e^{\mathbf{z}(i)}}{\sum_j e^{\mathbf{z}(j)}} \\ L &= -\sum_i \mathbf{y}(i) \ln \hat{\mathbf{y}}(i)\end{aligned}\quad (6.16)$$

反向传播的逐层计算公式为

$$\begin{aligned}\nabla_{\mathbf{z}} L &= \hat{\mathbf{y}} - \mathbf{y} \\ \nabla_{\mathbf{W}^{(2)}} L &= \mathbf{h} \nabla_{\mathbf{z}} L^T \\ \nabla_{\mathbf{b}^{(2)}} L &= \nabla_{\mathbf{z}} L \\ \nabla_{\mathbf{h}} L &= \mathbf{W}^{(2)T} \nabla_{\mathbf{z}} L \\ \nabla_{\mathbf{W}^{(1)}} L &= \mathbf{x} \nabla_{\mathbf{h}} L^T \\ \nabla_{\mathbf{b}^{(1)}} L &= \nabla_{\mathbf{h}} L \\ \nabla_{\mathbf{x}} L &= \mathbf{W}^{(1)T} \nabla_{\mathbf{h}} L\end{aligned}\quad (6.17)$$

其中 $\nabla_{\mathbf{z}} L$ 、 $\nabla_{\mathbf{h}} L$ 、 $\nabla_{\mathbf{x}} L$ 分别是损失函数对 Softmax 层、第二层、第一层的偏导， $\nabla_{\mathbf{W}^{(2)}} L$ 、 $\nabla_{\mathbf{b}^{(2)}} L$ 、 $\nabla_{\mathbf{W}^{(1)}} L$ 、 $\nabla_{\mathbf{b}^{(1)}} L$ 分别是第二层和第一层的权重和偏置梯度， η 为学习率。更新两个全连接层的权重和偏置的计算为

$$\begin{aligned}\mathbf{W}^{(1)} &\leftarrow \mathbf{W}^{(1)} - \eta \nabla_{\mathbf{W}^{(1)}} L \\ \mathbf{b}^{(1)} &\leftarrow \mathbf{b}^{(1)} - \eta \nabla_{\mathbf{b}^{(1)}} L \\ \mathbf{W}^{(2)} &\leftarrow \mathbf{W}^{(2)} - \eta \nabla_{\mathbf{W}^{(2)}} L \\ \mathbf{b}^{(2)} &\leftarrow \mathbf{b}^{(2)} - \eta \nabla_{\mathbf{b}^{(2)}} L\end{aligned}\quad (6.18)$$

神经网络训练相关的详细介绍可以参见《智能计算系统》教材第 2.2 节。

6.2.2.3 精度评估

在图像分类任务中，通常使用测试集的平均分类正确率来判断分类结果的精度。假设共有 N 个图像样本，例如 MNIST 手写数据集中共包含 10000 张测试图像，此时 $N = 10000$ 。神经网络推理出的第 i 张图像的分类结果为 $\hat{\mathbf{C}}(i)$ ，第 i 张图像的标记（即实际类别）为 $\mathbf{C}(i)$ ，则平均分类正确率 R 的计算公式为

$$R = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{\mathbf{C}}(i) = \mathbf{C}(i)) \quad (6.19)$$

其中 $\mathbf{1}(\hat{\mathbf{C}}(i) = \mathbf{C}(i))$ 表示当第 i 张图像的预测出的类别与标记相等时值为 1，否则值为 0。

6.2.2.4 CNL 算子库的 Python 封装

深度学习编程库 pycnnl 通过调用 MLU 上的 CNL 库中的高性能算子实现了全连接层、卷积层、池化层、ReLU 激活层、Softmax 损失层等常用的网络层的基本功能，并提供了常用网络层的 Python 接口。

pycnnl 提供的编程接口可以用于在 MLU 上加速神经网络算法,具体接口说明如表6.1所示。

pycnnl 用 Python 封装了一个 C++ 类 CnnlNet, 该类的成员函数定义了神经网络中层的创建、网络前向传播、参数加载等操作。

表 6.1 pycnnl 接口说明

接口	功能描述	参数/返回值
setInputShape: pycnnl.CnnlNet().setInputShape(dim_1, dim_2, dim_3, dim_4)	设定网络第一层输入数据的形状	dim_1 (int): 维度 1 dim_2 (int): 维度 2 dim_3 (int): 维度 3 dim_4 (int): 维度 4
createConvLayer: pycnnl.CnnlNet().createConvLayer(layer_name, input_shape, output_channel, kernel_size, stride, dilation, pad)	创建卷积层	layer_name: 该层的名称 input_shape(int list): 输入数据的形状,[N,input channel,input height,input width] output_channel (int): 输出 channel 的大小 kernel_size (int): 卷积核的大小 stride (int): 卷积步长 dilation (int): 膨胀系数 pad (int): 填充大小
createMlpLayer: pycnnl.CnnlNet().createMlpLayer(layer_name, input_shape, weight_shape, output_shape)	创建全连接层	layer_name: 该层的名称 input_shape(int list): 输入数据的形状, [N,input height,input width,input channel] weight_shape (int list): 输入数据的形状, [N,weight height,input channel,output channel] output_shape (int list): 输入数据的形状, [N,output weight,output width,output channel]
createReLuLayer: pycnnl.CnnlNet().createReLuLayer(layer_name)	创建 ReLU 激活函数层	layer_name: 该层的名称
createSoftmaxLayer: pycnnl.CnnlNet().createSoftmaxLayer(layer_name, input_shape, axis=1)	创建 Softmax 损失层	layer_name: 该层的名称 input_shape (int list): 输入数据的形状: batch_size, 1, output_classes axis (int): 进行 Softmax 计算的维度
createPoolingLayer: pycnnl.CnnlNet().createPoolingLayer(layer_name, input_shape, kernel_size, stride)	创建最大池化层	layer_name: 该层的名称 input_shape (int list): 输入数据的形状 kernel_size (int): pool 窗口的大小 stride (int): 窗口滑动步长
loadParams: pycnnl.CnnlNet().loadParams(layer_id, filter_data, bias_data)	为指定的层加载参数	layer_id (int): 需要加载权重的层的 id。CnnlNet 中将创建的层存储在一个数组中, id 即为当前层在该数组中的下标, 比如第一个层的 id 为 0, 第二个层的 id 则为 1 filter_data (list): 权重数据。必须是一维数组 bias_data(list): 参数偏置
setInputData: pycnnl.CnnlNet().setInputData(input_data)	加载输入数据	input_data (list): 输入数据。必须是一维数组, 数据布局为 NHWC
forward: pycnnl.CnnlNet().forward()	进行前向传播计算	
getOutputData: pycnnl.CnnlNet().getOutputData()	获取网络的计算结果	返回值 output_data: 网络最后一层的计算结果
size: pycnnl.CnnlNet().size()	获取当前神经网络中层的数量	返回值 layers_num (int): 当前网络中层的数量

下面以代码示例6.1为例，介绍如何调用 pycnnl 提供的编程接口来创建网络层。首先实例化 pycnnl.CnnlNet()，然后调用 CnnlNet 中的 createXXXLayer 成员函数就可以创建相应的网络层，例如创建全连接层时只需调用 pycnnl.CnnlNet().createMlpLayer。所有创建好的层对象的指针会按顺序以数组的形式保存在 CnnlNet 中，数组的下标作为层的 id 使用，当调用 pycnnl.CnnlNet().loadParams 函数时，便可以通过此 id 来指定需要加载参数的层。pycnnl.CnnlNet().forward 函数会遍历层数组中的对象，依次调用每个网络层的前向传播函数，最终返回最后一层的前向传播结果。

代码示例 6.1 pycnnl 创建层程序示例

```
1 # 实例化 CnnlNet
2 net = pycnnl.CnnlNet()
3 # 设定神经网络输入维度
4 net.setInputShape(1, 3, 224, 224)
5 # conv1_1
6 # 创建卷积层
7 input_shape1 = pycnnl.IntVector(4)
8 input_shape1[0] = 1
9 input_shape1[1] = 3
10 input_shape1[2] = 224
11 input_shape1[3] = 224
12 net.createConvLayer('conv1_1', input_shape1, 64, 3, 1, 1, 1)
13 # relu1_1
14 net.createReLuLayer('relu1_1')
```

感兴趣的同学可以进一步阅读 pycnnl 源码中网络层的实现，了解如何调用 CNL 库中的高性能算子实现全连接层的基本功能，ReLU 层和 Softmax 层的底层实现与之类似，具体每一层的 C++ 代码可以在 pycnnl/src/layers 中查看。

6.2.3 实验环境

硬件平台：MLU 云平台环境。

软件环境：PyTorch 框架，CNL 与 Torch-MLU-OPS 加速库，Python 3 及 Pillow、SciPy、NumPy 等扩展库。

数据集：MNIST 手写数字库^[78]。该数据集包含一个训练集和一个测试集，其中训练集有 60000 个样本，测试集有 10000 个样本。每个样本都由灰度图像（即单通道图像）及其标记组成，每个样本图像的大小为 28×28。MNIST 数据集包含 4 个文件，分别是训练集图像、训练集标记、测试集图像、测试集标记。下载地址为<http://yann.lecun.com/exdb/mnist/>。

6.2.4 实验内容

本实验基于 CPU 和 MLU 两种平台，设计并实现一个三层全连接神经网络，用于手写数字图像的分类。神经网络包含两个可调节的隐层和一个输出层，输入由图像数据决定，输出对应图像所属的类别，如 0-9 共 10 类。图像在输入前需转为一维向量，网络结构和训练参数均可根据实际需求灵活设置。

为实现可复用、易扩展的工程结构，本实验采用模块化方式组织，主要包括以下五个模块：

1) 数据加载模块：从文件中读取数据，并进行预处理，包括归一化、维度变换等处理。如果需要人为对数据进行随机数据扩增，则数据扩增处理也在数据加载模块中实现。

2) 基本单元模块：实现神经网络中不同类型的网络层的定义、前向传播计算、反向传播计算等功能。

3) 网络结构模块：利用基本单元模块建立一个完整的神经网络。

4) 网络训练模块：该模块实现用训练集进行神经网络训练的功能。在已建立的神经网络结构基础上，实现神经网络的前向传播、神经网络的反向传播、神经网络参数更新、神经网络参数保存等基本操作，以及训练函数主体。

5) 网络推理模块：该模块实现使用训练得到的网络模型，对测试样本进行预测的过程。具体实现的操作包括训练得到的网络模型参数的加载、神经网络的前向传播等。

本实验采用上述较为细致的模块划分方式。需要说明的是，目前有些开源的神经网络工程可能采用较粗的模块划分方式，例如将基本单元模块与网络结构模块合并，或将网络训练模块与网络推理模块合并。在某些特殊的应用场景下，神经网络工程可能不需要包含所有模块。

6.2.5 三层神经网络 CPU 训练的实验步骤

本节介绍如何实现训练三层神经网络涉及的各个模块，以及如何搭建和调用各个模块来实现手写数字图像分类。

6.2.5.1 数据加载模块

本实验采用的数据集是 MNIST 手写数字库^[78]。该数据集中的图像数据和标记数据采用表6.2中的 IDX 文件格式存放。图像的像素值按行优先顺序存放，取值范围为 [0,255]，其中 0 表示黑色，255 表示白色。

表 6.2 MNIST 数据集 IDX 文件格式^[78]

图像文件格式			
字节偏移	数据类型	值	描述
0000	int32 (32 位有符号整型)	0x00000803(2051)	magic number (魔数): 表示像素的数据类型以及像素数据的维度信息, MSB (大尾端)
0004	int32	60000 (训练集) 10000 (测试集)	图像数量
0008	int32	28	图像行数, 即图像高度
0012	int32	28	图像列数, 即图像宽度
0016	uint8 (8 位无符号整型)	??	像素值
0017	uint8	??	像素值
.....			
xxxx	uint8	??	像素值
标记文件格式			
字节偏移	数据类型	值	描述
0000	int32	0x00000801(2049)	magic number
0004	int32	60000 (训练集) 10000 (测试集)	标记数量
0008	uint8	??	标记
0009	uint8	??	标记
.....			
xxxx	uint8	??	标记

首先编写读取 MNIST 数据集文件并预处理的子函数, 如代码示例6.2所示。

代码示例 6.2 MNIST 数据集文件的读取和预处理

```

1 #` file: mnist\mlp\_cpu.py`
2 def load_mnist(self, file_dir, is_images = 'True'):
3     bin_file = open(file_dir, 'rb')
4     bin_data = bin_file.read()
5     bin_file.close()
6     if is_images: # 读取图像数据
7         fmt_header = '>iiii'
8         magic, num_images, num_rows, num_cols = struct.unpack_from(fmt_header, bin_data, 0)
9     else: # 读取标记数据
10        fmt_header = '>ii'
11        magic, num_images = struct.unpack_from(fmt_header, bin_data, 0)
12        num_rows, num_cols = 1, 1
13    data_size = num_images * num_rows * num_cols
14    mat_data = struct.unpack_from('>' + str(data_size) + 'B', bin_data, struct.calcsize(fmt_header))
15    mat_data = np.reshape(mat_data, [num_images, num_rows * num_cols])
16    return mat_data

```

然后调用该子函数对 MNIST 数据集中的 4 个文件分别进行读取和预处理, 并将处理过的训练和测试数据存储在 NumPy 矩阵中, 使训练模型时可以快速读取该矩阵中的数据, 如代码示例6.3所示。

代码示例 6.3 MNIST 子数据集的读取和预处理

```

1 #` file: mnist\mlp\_cpu.py`
2 def load_data(self):
3     #` TODO: 调用函数load\_mnist读取和预处理MNIST中训练数据和测试数据的图像和标记`
4     train_images = self.load_mnist(os.path.join(MNIST_DIR, TRAIN_DATA), True)
5     train_labels = _____

```

```

6 test_images = _____
7 test_labels = _____
8 self.train_data = np.append(train_images, train_labels, axis=1)
9 self.test_data = np.append(test_images, test_labels, axis=1)

```

6.2.5.2 基本单元模块

本实验采用图6.1所示的三层神经网络，主体是三个全连接层。在前两个全连接层之后均使用 ReLU 激活函数层引入非线性变换，在神经网络的最后添加 Softmax 层计算交叉熵损失。因此，本实验中需要实现的基本单元模块包括全连接层、ReLU 激活函数层和 Softmax 损失层。

在神经网络实现中，通常同类型的层用一个类来定义，多个同类型的层用类的实例来实现，层内的计算用类的成员函数来定义。类的成员函数通常包括层的初始化、参数的初始化、前向传播计算、反向传播计算、参数的更新、参数的加载和保存等。其中层的初始化函数一般会根据实例化层时的输入确定该层的超参数，例如该层的输入神经元数量和输出神经元数量等；参数的初始化函数会对该层的参数（如全连接层中的权重和偏置）定义变量，并填充初始值；前向传播函数利用前一层的输出作为本层的输入，计算本层的输出结果；反向传播函数根据链式法则逆序逐层计算损失函数对权重和偏置的梯度；参数的更新函数利用反向传播函数计算的梯度对本层的参数进行更新；参数的加载函数从给定的文件中加载参数的值，参数的保存函数将当前层参数的值保存到指定的文件中。有些层，如激活函数层，可能没有参数，就不需要定义参数的初始化、更新、加载和保存函数。有些层，如激活函数层和损失函数层，其输出维度由输入维度决定，不需要人工设定，因此不需要层的初始化函数。

以下是全连接层、ReLU 激活函数层和 Softmax 损失层的具体实现步骤。

1) **全连接层**：实现如代码示例6.4所示，定义了以下成员函数：

- 层的初始化：需要确定该全连接层的输入神经元个数，即输入二维矩阵中每个行向量的维度，和输出神经元个数，即输出二维矩阵中每个行向量的维度。
- 参数初始化：全连接层的参数包括权重和偏置。根据输入神经元个数 m 和输出神经元个数 n 可以确定权重 $\mathbf{W}$ 的维度为 $m \times n$ ，偏置 $\mathbf{b}$ 的维度为 n 。在对权重和偏置进行初始化时，通常利用高斯随机数来初始化权重的值，而将偏置的所有值初始化为 0。
- 前向传播计算：全连接层的前向传播计算公式为公式(6.3)，可以通过输入矩阵与权重矩阵相乘再与偏置相加实现。
- 反向传播计算：全连接层的反向传播计算公式为公式(6.4)。给定损失函数对本层输出的偏导 $\nabla_{\mathbf{Y}} L$ ，利用矩阵相乘计算权重和偏置的梯度 $\nabla_{\mathbf{W}} L$ 、 $\nabla_{\mathbf{b}} L$ 以及损失函数对本层输入的偏导 $\nabla_{\mathbf{X}} L$ 。
- 参数更新：给定学习率 η ，利用反向传播计算得到的权重梯度 $\nabla_{\mathbf{W}} L$ 和偏置梯度 $\nabla_{\mathbf{b}} L$ 对本层的权重 $\mathbf{W}$ 和偏置 $\mathbf{b}$ 进行更新：

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} L \quad (6.20)$$

$$\mathbf{b} \leftarrow \mathbf{b} - \eta \nabla_{\mathbf{b}} L \quad (6.21)$$

- 参数加载：从该函数的输入中读取本层的权重 W 和偏置 b 。
- 参数保存：返回本层当前的权重 W 和偏置 b 。

代码示例 6.4 全连接层的实现

```

1 #` file: layers\_l.py`
2 class FullyConnectedLayer(object):
3     def __init__(self, num_input, num_output): # 全连接层初始化
4         self.num_input = num_input
5         self.num_output = num_output
6     def init_param(self, std=0.01): # 参数初始化
7         self.weight = np.random.normal(loc=0.0, scale=std, size=(self.num_input, self.num_output))
8         self.bias = np.zeros([1, self.num_output])
9     def forward(self, input): # 前向传播的计算
10        self.input = input
11        #` TODO: 全连接层的前向传播, 计算输出结果`
12        self.output = _____
13        return self.output
14    def backward(self, top_diff): # 反向传播的计算
15        #` TODO: 全连接层的反向传播, 计算参数梯度和本层损失`
16        self.d_weight = _____
17        self.d_bias = _____
18        bottom_diff = _____
19        return bottom_diff
20    def update_param(self, lr): # 参数更新
21        #` TODO: 利用梯度对全连接层参数进行更新`
22        self.weight = _____
23        self.bias = _____
24    def load_param(self, weight, bias): # 参数加载
25        self.weight = weight
26        self.bias = bias
27    def save_param(self): # 参数保存
28        return self.weight, self.bias

```

2) **ReLU 激活函数层**：ReLU 层不包含参数，因此实现中没有参数初始化、参数更新、参数的加载和保存相关的函数。ReLU 层的实现如代码示例6.5所示，定义了以下成员函数：

- 前向传播计算：根据公式(6.5)可以计算 ReLU 层前向传播的结果。在工程实现中，可以对整个输入矩阵使用 maximum 函数，maximum 函数会进行广播，计算输入矩阵的每个元素与 0 的最大值。
- 反向传播计算：根据公式(6.6)可以计算损失函数对输入的偏导。在工程实现中，可以获取 $x(i) < 0$ 的位置索引，将 $\nabla_x L$ 中对应位置的值置为 0。

代码示例 6.5 ReLU 激活函数层的实现，其中 TODO 表示需要读者来实现的内容

```

1 #` file: layers\_l.py`
2 class ReLULayer(object):
3     def forward(self, input): # 前向传播的计算
4         self.input = input
5         #` TODO: ReLU层的前向传播, 计算输出结果`
6         output = _____
7         return output
8     def backward(self, top_diff): # 反向传播的计算
9         #` TODO: ReLU层的反向传播, 计算本层损失`
10        bottom_diff = _____
11        return bottom_diff

```

3) **Softmax 损失层**：同样不包含参数，因此实现中没有参数初始化、更新、加载和保

存相关的函数。但该层需要计算总的损失函数值，作为训练时的中间输出结果，帮助判断模型的训练进程。Softmax 损失层的实现如代码示例6.6所示，定义了以下成员函数：

- 前向传播计算：可以使用公式(6.11)计算。该公式为确保数值稳定，会在求 e 指数前做减最大值处理。
- 损失函数计算：可以使用公式(6.12)计算，采用批量随机梯度下降法训练时损失值是 batch 内所有样本的损失值的均值。需要注意的是，MNIST 手写数字库的标记数据读入的是 0 至 9 的类别编号，在计算损失时需要先将类别编号转换为 one-hot 向量。
- 反向传播计算：可以使用公式(6.13)计算，计算时同样需要对样本数量取平均。

代码示例 6.6 Softmax 损失层的实现

```
1 #` file: layers\_l.py`
2 class SoftmaxLossLayer(object):
3     def forward(self, input): # 前向传播的计算
4         #` TODO: Softmax 损失层的前向传播, 计算输出结果`
5         input_max = np.max(input, axis=1, keepdims=True)
6         input_exp = np.exp(input - input_max)
7         self.prob = np.sum(input_exp, axis=1, keepdims=True)
8         return self.prob
9     def get_loss(self, label): # 计算损失
10        self.batch_size = self.prob.shape[0]
11        self.label_onehot = np.zeros_like(self.prob)
12        self.label_onehot[np.arange(self.batch_size), label] = 1.0
13        loss = -np.sum(np.log(self.prob) * self.label_onehot) / self.batch_size
14        return loss
15    def backward(self): # 反向传播的计算
16        #` TODO: Softmax 损失层的反向传播, 计算本层损失`
17        bottom_diff = np.zeros_like(self.prob)
18        return bottom_diff
```

6.2.5.3 网络结构模块

网络结构模块利用已经实现的神经网络的基本单元来建立一个完整的神经网络。在工程实现中通常用一个类来定义一个神经网络，用类的成员函数来定义神经网络的初始化、建立神经网络结构、神经网络参数初始化等基本操作。本实验中三层神经网络的网络结构模块的实现如代码示例6.7所示，定义了以下成员函数：

- 神经网络初始化：确定神经网络相关的超参数，例如网络中每个隐层的神经元个数。
- 建立网络结构：定义整个神经网络的拓扑结构，实例化基本单元模块中定义的层并将这些层进行堆叠。例如本实验使用的三层神经网络包含三个全连接层，并且在前两个全连接层后都跟随有 ReLU 层，神经网络的最后使用了 Softmax 损失层。
- 神经网络参数初始化：对于神经网络中包含参数的层，依次调用这些层的参数初始化函数，从而完成整个神经网络的参数初始化。本实验使用的三层神经网络中，只有三个全连接层包含参数，依次调用其参数初始化函数即可。

代码示例 6.7 三层神经网络的网络结构模块实现

```
1 #` file: mnist\_mlp\_cpu.py`
2 class MNIST_MLP(object):
3     def __init__(self, batch_size=100, input_size=784, hidden1=32, hidden2=16, out_classes=10, lr
4         =0.01, max_epoch=2, print_iter=100):
```

```

4      # 神经网络初始化
5      self.batch_size = batch_size
6      self.input_size = input_size
7      self.hidden1 = hidden1
8      self.hidden2 = hidden2
9      self.out_classes = out_classes
10     self.lr = lr
11     self.max_epoch = max_epoch
12     self.print_iter = print_iter
13 def build_model(self):
14     # TODO: 建立三层神经网络结构
15     print('Building multi-layer perception model...')
16     self.fc1 = _____
17     self.relu1 = _____
18     self.fc2 = _____
19     self.relu2 = _____
20     self.fc3 = _____
21     self.softmax = _____
22     self.update_layer_list = [self.fc1, self.fc2, self.fc3]
23 def init_model(self): # 神经网络参数初始化
24     for layer in self.update_layer_list:
25         layer.init_param()

```

6.2.5.4 网络训练模块

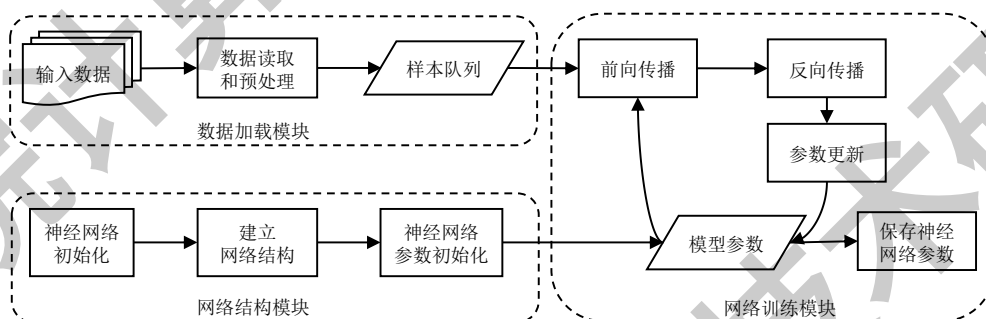


图 6.3 神经网络训练流程

神经网络训练流程如图6.3所示。在完成数据加载模块和网络结构模块实现之后，需要实现网络训练模块。本实验中三层神经网络的训练模块的实现如代码示例6.8所示。神经网络的训练模块通常拆解为若干步骤，包括神经网络的前向传播、神经网络的反向传播、神经网络参数更新、神经网络参数保存等基本操作。这些网络训练模块的基本操作以及训练主体用神经网络类的成员函数来定义：

- 神经网络的前向传播：根据神经网络的拓扑结构，顺序调用每层的前向传播函数。以输入数据作为第一层的输入，之后每层的输出作为其下一层的输入，顺序计算每一层的输出，最后得到损失函数层的输出。
- 神经网络的反向传播：根据神经网络的拓扑结构，逆序调用每层的反向传播函数。采用链式法则逆序逐层计算损失函数对每层参数的偏导，最后得到神经网络所有层的参数梯度。
- 神经网络参数更新：对神经网络中包含参数的层，依次调用各层的参数更新函数，来对整个神经网络的参数进行更新。本实验使用的三层神经网络中只有三个全连接层包含参

数，因此依次更新三个全连接层的参数即可。

- 神经网络参数保存：对神经网络中包含参数的层，依次收集这些层的参数并存储到文件中。

- 神经网络训练主体：在该函数中，(1) 确定训练的一些超参数，如使用批量梯度下降算法时的批量大小、学习率大小、迭代次数或训练周期次数、可视化训练过程时每迭代多少次屏幕输出一次当前的损失值等等。(2) 开始迭代训练过程。每次迭代训练开始前，可以根据需要对数据进行随机打乱，一般是一个训练周期后对数据集进行随机打乱，其中当整个数据集的数据都参与一次训练过程为完成一个训练周期。每次迭代训练过程中，先选取当前迭代所使用的数据和对应的标记，再进行整个网络的前向传播，随后计算当前迭代的损失值，然后进行整个网络的反向传播来获得整个网络的参数梯度，最后对整个网络的参数进行更新。完成一次迭代后可以根据需要在屏幕上输出当前的损失值，以供实际应用中修改模型参考。完成神经网络的训练过程后，通常会将训练得到的神经网络模型参数保存到文件中。

代码示例 6.8 三层神经网络的训练模块实现

```
1  #` file: mnist_mlp_cpu.py`
2  def forward(self, input):
3      # TODO: 神经网络的前向传播
4      h1=self.fc1.forward(input)
5      h1=self.relu1.forward(h1)
6      ###fc2#
7      h2=_____
8      h2=_____
9      ###fc3##
10     h3=_____
11     prob=_____
12     return prob
13
14 def backward(self):
15     # TODO: 神经网络的反向传播
16     dloss = self.softmax.backward()
17     #####
18     dh3 = _____
19     dh2 = _____
20     dh2 = _____
21     dh1 = _____
22     dh1 = self.fc1.backward(dh1)
23
24 def update(self, lr): # 神经网络参数更新
25     for layer in self.update_layer_list:
26         layer.update_param(lr)
27
28 def save_model(self, param_dir): # 保存神经网络参数
29     params = {}
30     params['w1'], params['b1'] = self.fc1.save_param()
31     params['w2'], params['b2'] = self.fc2.save_param()
32     params['w3'], params['b3'] = self.fc3.save_param()
33     np.save(param_dir, params)
34
35 def train(self): # 训练函数主体
36     max_batch = self.train_data.shape[0] // self.batch_size
37     for idx_epoch in range(self.max_epoch):
38         mlp.shuffle_data()
39         for idx_batch in range(max_batch):
```

```

40     batch_images = self.train_data[idx_batch*self.batch_size:(idx_batch+1)*self.batch_size,
41                                     :-1]
42     batch_labels = self.train_data[idx_batch*self.batch_size:(idx_batch+1)*self.batch_size,
43                                     -1]
44     prob = self.forward(batch_images)
45     loss = self.softmax.get_loss(batch_labels)
46     self.backward()
47     self.update(self.lr)
48     if idx_batch % self.print_iter == 0:
49         print('Epoch %d, iter %d, loss: %.6f' % (idx_epoch, idx_batch, loss))

```

6.2.5.5 网络推理模块

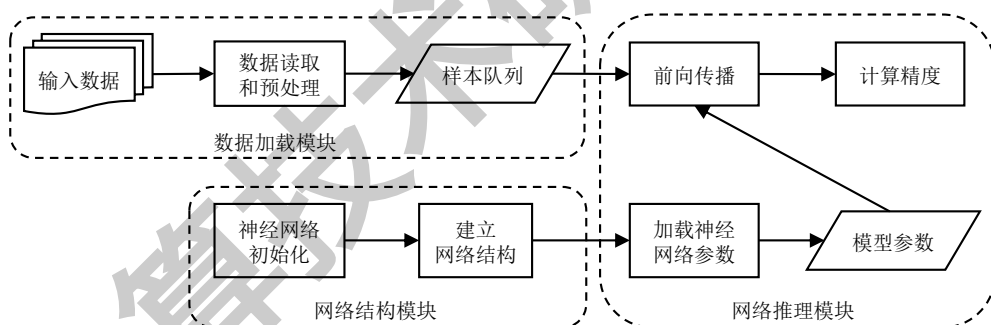


图 6.4 神经网络推理流程

整个神经网络推理流程如图6.4所示。完成神经网络的训练之后，可以用训练得到的模型对测试数据进行预测，以评估模型的精度。本实验中三层神经网络的推理模块的实现如代码示例6.9所示。工程实现中同样常将一个神经网络的推理模块拆解为若干步骤，包括神经网络参数加载、神经网络的前向传播等基本操作。这些网络推理模块的基本操作以及推理主体用神经网络类的成员函数来定义：

- 神经网络的前向传播：网络推理模块中的神经网络前向传播操作与网络训练模块中的前向传播操作完全一致，因此可以直接调用网络训练模块中的神经网络前向传播函数。
- 神经网络参数加载：读取神经网络训练模块保存的模型参数文件，并加载有参数的网络层的参数值。
- 神经网络推理函数主体：在进行神经网络推理前，需要从模型参数文件中加载神经网络的参数。在神经网络推理过程中，循环每次读取一定批量的测试数据，随后进行整个神经网络的前向传播计算得到神经网络的输出结果，对于每个样本取输出结果中最大概率对应的类别作为预测的分类类别，再与测试数据集的标记进行比对，利用相关的评价函数计算模型的精度，如手写数字分类问题使用分类平均正确率作为模型的评价函数。

代码示例 6.9 三层神经网络的推理模块实现

```

1 # file: mnist_mlp_cpu.py
2 def load_model(self, param_dir): # 加载神经网络参数
3     params = np.load(param_dir, allow_pickle=True, encoding="latin1").item()
4     self.fc1.load_param(params['w1'], params['b1'])
5     self.fc2.load_param(params['w2'], params['b2'])
6     self.fc3.load_param(params['w3'], params['b3'])
7

```

```

8 def evaluate(self): # 推理函数主体
9     pred_results = np.zeros([self.test_data.shape[0]])
10    for idx in range(self.test_data.shape[0]//self.batch_size):
11        batch_images = self.test_data[idx*self.batch_size:(idx+1)*self.batch_size, :-1]
12        prob = self.forward(batch_images)
13        pred_labels = np.argmax(prob, axis=1)
14        pred_results[idx*self.batch_size:(idx+1)*self.batch_size] = pred_labels
15    accuracy = np.mean(pred_results == self.test_data[:, -1])
16    print('Accuracy in test set: %f' % accuracy)

```

6.2.5.6 完整实验流程

完成神经网络的各个模块之后,就可以调用这些模块实现用三层神经网络进行手写数字图像分类的完整流程,如代码示例6.10所示。首先实例化三层神经网络对应的类,指定神经网络的超参数,如每层的神经元个数。其次进行数据的加载和预处理。再调用网络结构模块建立神经网络,随后进行网络初始化,在该过程中网络结构模块会自动调用基本单元模块实例化神经网络中的每个层。然后调用网络训练模块训练整个网络,之后将训练得到的模型参数保存到文件中。最后从文件中读取训练得到的模型参数,调用网络推理模块测试网络的精度。

代码示例 6.10 三层神经网络的完整流程实现

```

1 #` file: mnist_mlp_cpu.py`
2 def build_mnist_mlp(param_dir='weight.npy'):
3     h1, h2, e = 32, 16, 10
4     mlp = MNIST_MLP(hidden1=h1, hidden2=h2, max_epoch=e)
5     mlp.load_data()
6     mlp.build_model()
7     mlp.init_model()
8     mlp.train()
9     mlp.save_model('mlp-%d-%d-%depoch.npy' % (h1, h2, e))
10    mlp.load_model('mlp-%d-%d-%depoch.npy' % (h1, h2, e))
11    return mlp
12
13 if __name__ == '__main__':
14     mlp = build_mnist_mlp()
15     mlp.evaluate()

```

6.2.5.7 实验运行

根据第6.2.5.1节 ~ 第6.2.5.6节的描述补全 layer_1.py、mnist_mlu_cpu.py 代码,并通过 Python 运行.py 代码。具体可以参考以下步骤。

1) 环境申请

申请实验环境并登录云平台,/opt/code_chap_6/exp_6_1_NNclassification/exp_6_1_1_traincpu/目录存放的是本实验的示例代码。

```

# 登录云平台
ssh root@xxx.xxx.xxx -p xxxxx
# 进入 /opt/code_chap_6/exp_6_1_NNclassification/exp_6_1_1_traincpu/ 目录
cd /opt/code_chap_6/exp_6_1_NNclassification/exp_6_1_1_traincpu/

```

2) 代码实现

如下所示，补全 `stu_upload` 中的 `layers_1.py`, `mnist_mlp_cpu.py` 文件。

```
# 补全 layers_1.py, mnist_mlp_cpu.py
vim stu_upload/layers_1.py
vim stu_upload/mnist_mlp_cpu.py
```

3) 运行实验

直接执行主函数文件即可运行本实验。

```
# 运行完整实验
python main_train_cpu.py
```

6.2.5.8 实验评估

本实验的评估标准设定如下：

- 60 分标准：给定全连接层、ReLU 层、Softmax 损失层的前向传播的输入矩阵、参数值、反向传播的输入，可以得到正确的前向传播的输出矩阵、反向传播的输出和参数梯度。
- 80 分标准：实现正确的三层神经网络，并进行训练和推理，使最后训练得到的模型在 MNIST 测试数据集上的平均分类正确率高于 92%。
- 90 分标准：实现正确的三层神经网络，并进行训练和推理，通过调整与训练相关的超参数，使最后训练得到的模型在 MNIST 测试数据集上的平均分类正确率高于 95%。
- 100 分标准：在三层神经网络基础上设计自己的神经网络结构，并进行训练和推理，使最后训练得到的模型在 MNIST 测试数据集上的平均分类正确率高于 98%。

6.2.5.9 实验思考

- 1) 在实现神经网络基本单元时，如何确保一个网络层的实现是正确的？
- 2) 在实现神经网络后，如何在改变网络结构的条件下提高精度？
- 3) 如何通过修改网络结构提高精度？可以从哪些方面修改网络结构？

6.2.6 三层神经网络 MLU 推理的实验步骤

6.2.6.1 数据加载模块

本实验采用的数据集依然是 MNIST 手写数字库，且数据读取的函数与第 6.2.5.1 节的实现相同。因为本实验只完成推理功能，因此只需读取测试数据，对其进行预处理后存储在 NumPy 矩阵中，方便后续推理时快速读取数据。数据加载模块的实现如代码示例 6.11 所示。

代码示例 6.11 MNIST 子数据集的读取和预处理

```
1 #` file: mnist\_mlp\_demo.py`
2 def load_data(self, data_path, label_path):
3     #` TODO: 调用函数load\_mnist读取和预处理MNIST中测试数据的图像和标记`
4     test_images = _____
5     test_labels = _____
6     self.test_data = np.append(test_images, test_labels, axis=1)
```


6.2.6.2 基本单元模块

pycnnl 库已经将常用网络层的实现用 Python 语言封装起来，因此可以直接调用 pycnnl 中的相关 Python 接口来实现神经网络的基本单元模块。具体调用接口和方式可以参照表6.1和代码示例6.1。

6.2.6.3 网络结构模块

网络结构模块可以直接使用 pycnnl 封装好的基本单元接口来搭建一个完整的神经网络。在工程实现中，首先用一个类来定义一个神经网络，然后用类的成员函数来定义神经网络的初始化、建立神经网络结构等基本操作。MLU 上实现的网络结构模块如代码示例6.12所示。

代码示例 6.12 三层神经网络的网络结构模块的 MLU 实现

```

1  #` file : mnist\_mlp\_demo.py`
2  class MNIST_MLP(object):
3      def __init__(self):
4          # 初始化网络，创建pycnnl.CnnlNet() 实例
5          self.net = pycnnl.CnnlNet()
6
7      def build_model(self, batch_size=10000, input_size=784,
8                      hidden1=32, hidden2=16, out_classes=10):
9          self.batch_size = batch_size
10         self.out_classes = out_classes
11         # creating layers
12         self.net.setInputShape(batch_size, input_size, 1, 1) #设置输入参数
13         # fc1
14         input_shapem1=pycnnl.IntVector(4)
15         input_shapem1[0]=batch_size
16         input_shapem1[1]=1
17         input_shapem1[2]=1
18         input_shapem1[3]=input_size
19         weight_shapem1=pycnnl.IntVector(4)
20         weight_shapem1[0]=batch_size
21         weight_shapem1[1]=1
22         weight_shapem1[2]=input_size
23         weight_shapem1[3]=hidden1
24
25         output_shapem1=pycnnl.IntVector(4)
26         output_shapem1[0]=batch_size
27         output_shapem1[1]=1
28         output_shapem1[2]=1
29         output_shapem1[3]=hidden1
30
31         self.net.createMlpLayer('fc1', input_shapem1, weight_shapem1, output_shapem1)
32
33         # relu1
34         -----
35         # fc2
36         -----
37         # relu2
38         -----
39         # fc3
40         -----
41         # softmax
42         -----

```

在该示例程序中定义了以下成员函数：

- 网络初始化：创建 `pycnnl.CnnlNet()` 的实例 `net`。后续神经网络层的创建、参数的加载、前向传播计算等操作都通过该对象来调用。
- 建立网络结构：首先加载输入数据和模型权重的参数文件，然后定义整个神经网络的拓扑结构。定义网络结构时，使用 `net` 中的 `createXXXLayer` 函数来实例化每一层。

6.2.6.4 网络推理模块

搭建好网络后，就可以加载训练好的模型、输入数据进行预测。网络推理模块的 MLU 实现如代码示例6.13所示。

代码示例 6.13 三层神经网络的网络推理模块的 MLU 实现示例

```
1 #` file: mnist\_mlp\_demo.py`
2 def load_model(self, param_dir):
3     #` TODO: 分别为三层全连接层加载参数`
4     params = np.load(param_dir, allow_pickle=True, encoding="latin1").item()
5     weight1 = params['w1'].flatten().astype(np.float)
6     bias1 = params['b1'].flatten().astype(np.float)
7     self.net.loadParams(0, weight1, bias1)
8     weight2 = params['w2'].flatten().astype(np.float)
9     bias2 = params['b2'].flatten().astype(np.float)
10
11     -----
12     weight3 = params['w3'].flatten().astype(np.float)
13     bias3 = params['b3'].flatten().astype(np.float)
14     -----
15 def forward(self): # 前向传播
16     return self.net.forward()
17
18 def evaluate(self):
19     pred_results = np.zeros([self.test_data.shape[0]])
20     # 读取一定批量的测试数据进行前向传播
21     for idx in range(self.test_data.shape[0]//self.batch_size):
22         batch_images = self.test_data[idx*self.batch_size:(idx+1)*self.batch_size, :-1]
23         data = batch_images.flatten().tolist()
24         # 加载输入数据
25         self.net.setInputData(data)
26         # 打印推理的时间
27         start = time.time()
28         self.forward()
29         end = time.time()
30         print('inferencing time: %f'%(end - start))
31         prob = self.net.getOutputData()
32         prob = np.array(prob).reshape((self.batch_size, self.out_classes))
33         pred_labels = np.argmax(prob, axis=1)
34         pred_results[idx*self.batch_size:(idx+1)*self.batch_size] = pred_labels
35     accuracy = np.mean(pred_results == self.test_data[:, -1])
36     print('Accuracy in test set: %f'% accuracy)
```

在该示例程序中，神经网络推理模块的参数加载、前向传播、精度计算等基本操作拆分为神经网络类的成员函数来定义：

- 神经网络的参数加载：读取模型参数文件，并使用 `net` 中的 `loadParams` 接口加载参数。可以使用第6.2节实验中训练得到的模型参数用于本实验，需要做如下处理：由于 Python 中的浮点数类型 `float` 是双精度浮点，`pycnnl` 接口内部实现的 C++ 函数接收的权重也只能是

双精度浮点数类型，而 numpy 存储的数据包括权重都是 np.float32 类型，因此需要手动将 numpy 数据类型转为 np.float64 类型，否则在调用 pycnnl 库的接口过程中会报错。

- 神经网络的前向传播：net.forward 函数会自动遍历调用 net 中的每一层的前向传播函数，并将最后一层前向传播的计算结果返回。

- 神经网络推理函数主体：与第6.2节中的 CPU 实现类似，循环读取一定批量的测试数据，随后调用网络的前向传播函数计算得到神经网络的输出结果，然后与测试数据集的标记进行比对计算得到模型的精度。

6.2.6.5 完整实验流程

完成所有模块的实现后，就可以调用上述模块中的函数，在 MLU 上运行神经网络实现手写数字图像分类。网络运行的流程与 CPU 上的执行流程基本一致。本实验中三层神经网络的完整流程的程序示例如代码示例6.14所示。首先实例化三层神经网络对应的类；其次调用网络结构模块 build_model 建立神经网络，指定神经网络的超参数（如每层的神经元个数）；随后调用 load_data 函数进行数据的加载和预处理；然后调用 load_model 函数从文件中读取训练好的模型参数；最后调用 evaluate 函数执行网络推理模块得到预测结果，并测试模型的精度。在上一节的 CPU 实验中，我们设置的默认 batch size 为 100。对于这种小规模运算，使用 MLU 这样算力强大的设备实在是有些大材小用了，因此我们将 batch size 改为 10000，一次传入 10000 张图片，记录 MLU 计算的时间。因为 CNNL 在第一次运行的时候会有一个指令生成的过程，导致运行时间会长一些，所以我们多次执行 evaluate 函数，排除第一次计算的时间，其它的每次计算时间就和真实的硬件时间很接近了。

用 load_data 和 load_model 加载数据和参数，CNNL 和 CNRT 会在 MLU 上分配内存空间，将参数拷入 MLU 的内存。

代码示例 6.14 三层神经网络的完整流程 MLU 实现示例

```
1 #` file: mnist\_mlp\_demo.py`
2 if __name__ == '__main__':
3     # 设置batch_size为10000
4     batch_size = 10000
5     h1, h2, c = 32, 16, 10
6     mlp = MNIST_MLP()
7     mlp.build_model(batch_size=batch_size, hidden1=h1, hidden2=h2, out_classes=c)
8     model_path = '../data/mnist_mlp_data/mlp-%d-%d-10epoch.npy'%(h1, h2)
9     test_data = '../data/mnist_mlp_data/mnist_data/t10k-images-idx3-ubyte'
10    test_label = '../data/mnist_mlp_data/mnist_data/t10k-labels-idx1-ubyte'
11    mlp.load_data(test_data, test_label)
12    mlp.load_model(model_path)
13    # 循环多次统计MLU计算时间
14    for i in range(3):
15        mlp.evaluate()
```

6.2.6.6 实验运行

根据第6.2.6.1节 ~ 第6.2.6.5节的描述补全 mnist_mlp_demo.py 代码，并通过 Python 运行.py 代码。具体可以参考以下步骤：

1) 环境申请

申请实验环境并登录云平台，`/opt/code_1_NNclassification/exp_6_1_2_inferdlp/`目录下存放是本实验的示例代码。

```
# 登录云平台
ssh root@xxx.xxx.xxx -p xxxxx
# 进入/opt/code_1_NNclassification/exp_6_1_2_inferdlp/目录
cd 云平台, /opt/code_1_NNclassification/exp_6_1_2_inferdlp/
```

2) 代码实现

补全 `stu_upload` 中的 `layers_1.py`, `mnist_mlp_cpu.py`, `mnist_mlp_demo.py` 文件。

```
# 补全实验代码
vim stu_upload/layers_1.py
vim stu_upload/mnist_mlp_cpu.py
vim stu_upload/mnist_mlp_demo.py
```

其中，需要将 `mnist_mlp_cpu.py` 文件中的 `build_mnist_mlp()` 函数按照 `readme.txt` 进行修改。

3) 运行实验

将实验6.2.5中训练得到的参数复制到 `stu_upload` 目录下,并将参数文件重命名为 `weight.npy`。然后运行实验:

```
# 运行完整实验
python main_infer_dlp.py
```

6.2.6.7 实验评估

本实验中，精度评判标准与第6.2节实验一样，使用测试集的平均分类正确率判断分类结果的精度。性能评判标准为设置 `batch size` 为 10000 时，进行一次 `forward` 的时间。本实验的评分标准设定如下：

- 60 分标准：完善本节实验代码，用 `pycnnl` 搭建出的三层神经网络能够在 `MLU` 上进行推理，并且在测试集上的平均分类正确率高于 90%。
- 80 分标准：修改网络的规模，使用第6.2节实验的代码重新训练模型，使训练得到的模型在 `MLU` 上运行的推理耗时为 `CPU` 推理耗时的 1/50 或更低，并且在测试集上的平均分类正确率高于 95%。
- 100 分标准：基于三层神经网络重新设计自己的神经网络结构，使用第6.2节实验的代码重新训练模型，使训练得到的模型在 `MLU` 上运行的推理耗时为 `CPU` 推理耗时的 1/200 或更低，并且在测试集上的平均分类正确率高于 98%。

6.2.6.8 实验思考

在实验中请思考如下问题：

- 1) `MLU` 在运行神经网络推理时，相对于 `CPU` 有什么优势和劣势？
- 2) 在什么样的神经网络结构下，`MLU` 能够最大化发挥它的性能优势？